

Backend Architecture

A Real-Time Messaging System Built for Scale and Security



Stacks Used



Node.js + TypeScript

Server runtime with type safety for robust, maintainable codebase



Socket.IO

Real-time bidirectional communication with automatic fallback handling



PostgreSQL

Relational database for structured data storage and complex queries



Express

Web framework for building RESTful API endpoints efficiently



Redis

In-memory cache for sessions, presence tracking, and rate limiting



Cloudinary

Media storage and transformation for images and files

Why These Stacks?

Node.js

Asynchronous, event-driven architecture, ideal for handling numerous concurrent connections required by real-time applications.

Express

Minimalist web framework that provides a robust set of features for building RESTful APIs efficiently and scaling with Node.js.

TypeScript

Enhances code quality and maintainability by adding static typing, catching errors early, and improving developer productivity.

Redis

In-memory data store for high-speed caching, real-time features like presence tracking, sessions, notifications, and efficient rate limiting.

Socket.IO

Enables real-time, bidirectional, and event-based communication between clients and the server with automatic fallback handling.

JWT + Argon2

JSON Web Tokens provide stateless authentication, while Argon2 ensures robust and secure hashing of user passwords.

What's Our Vision?

Modern chat app for campus with following features:



Real-Time Communication

Instant messaging with sub-second delivery across thousands of concurrent users



Secure Messaging

End-to-end encryption and comprehensive security measures protecting user data



Scalable Architecture

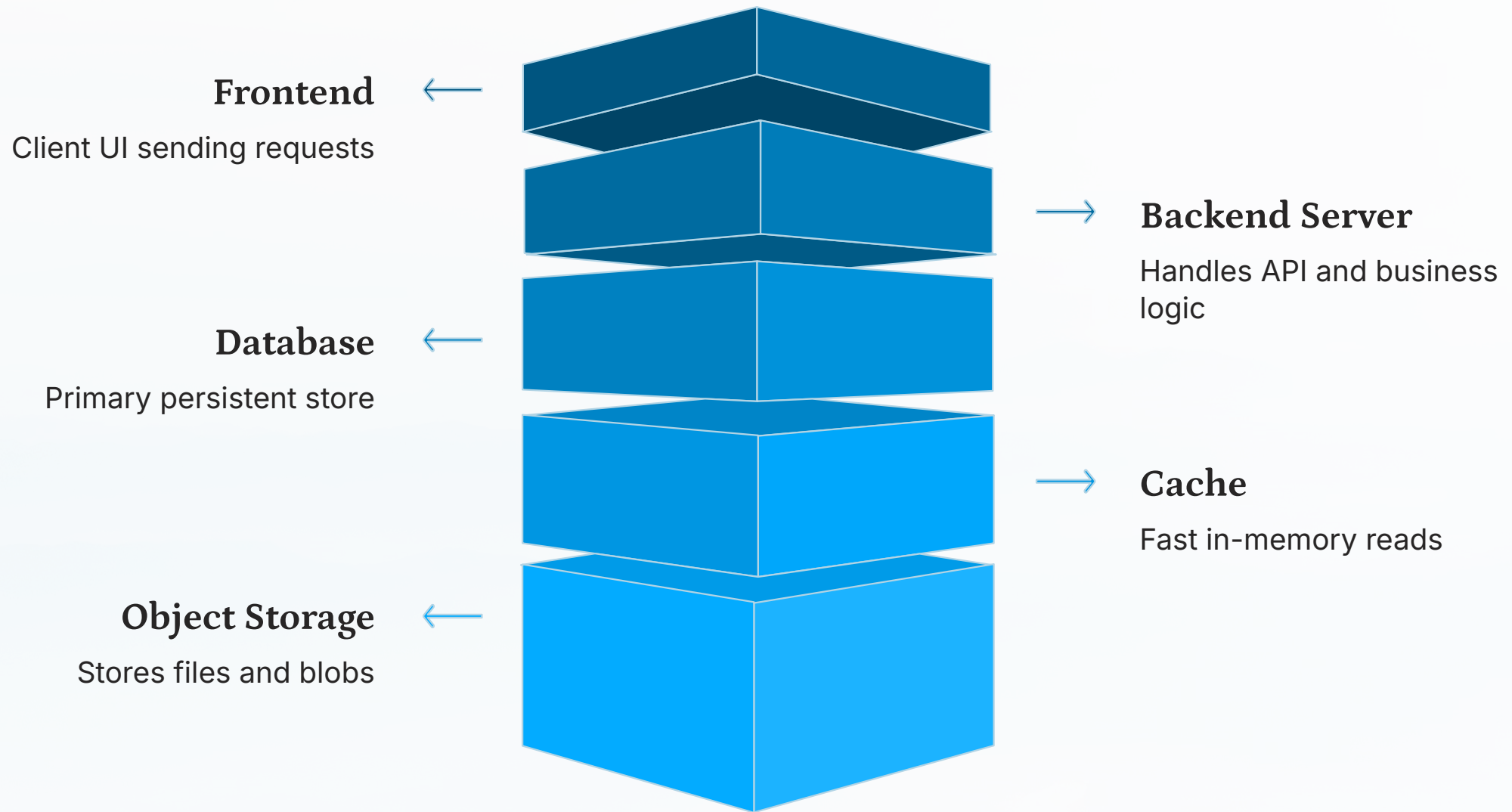
Horizontal scaling capabilities to handle millions of messages daily



Reliable Delivery

Guaranteed message persistence and delivery even during network disruptions

High Level Architecture



The three-tier architecture shows how the system is organized from the client-facing frontend through the backend server to the data layer

Backend Module Structure

Controllers

Request handling and response formatting

Services

Business logic and data processing

Middleware

Validation, authentication, and security

Routes

Endpoint definitions and request routing

Socket

Real-time event handling

Config+Utils

Environment Settings, Constants, Utilities

This separation of concerns ensures maintainable, testable code with clear responsibility boundaries.

Module Functions

Each module plays a distinct role in processing requests and managing data flow.

Controllers

Receive and process incoming API requests, orchestrating responses.

Services

Encapsulate business logic, performing operations and interacting with data sources.

Middleware

Intercept requests for tasks like authentication, validation, and logging before reaching controllers.

Routes

Define API endpoints and direct incoming requests to the appropriate controller methods.

Socket

Manage real-time communication events and broadcast updates to connected clients.

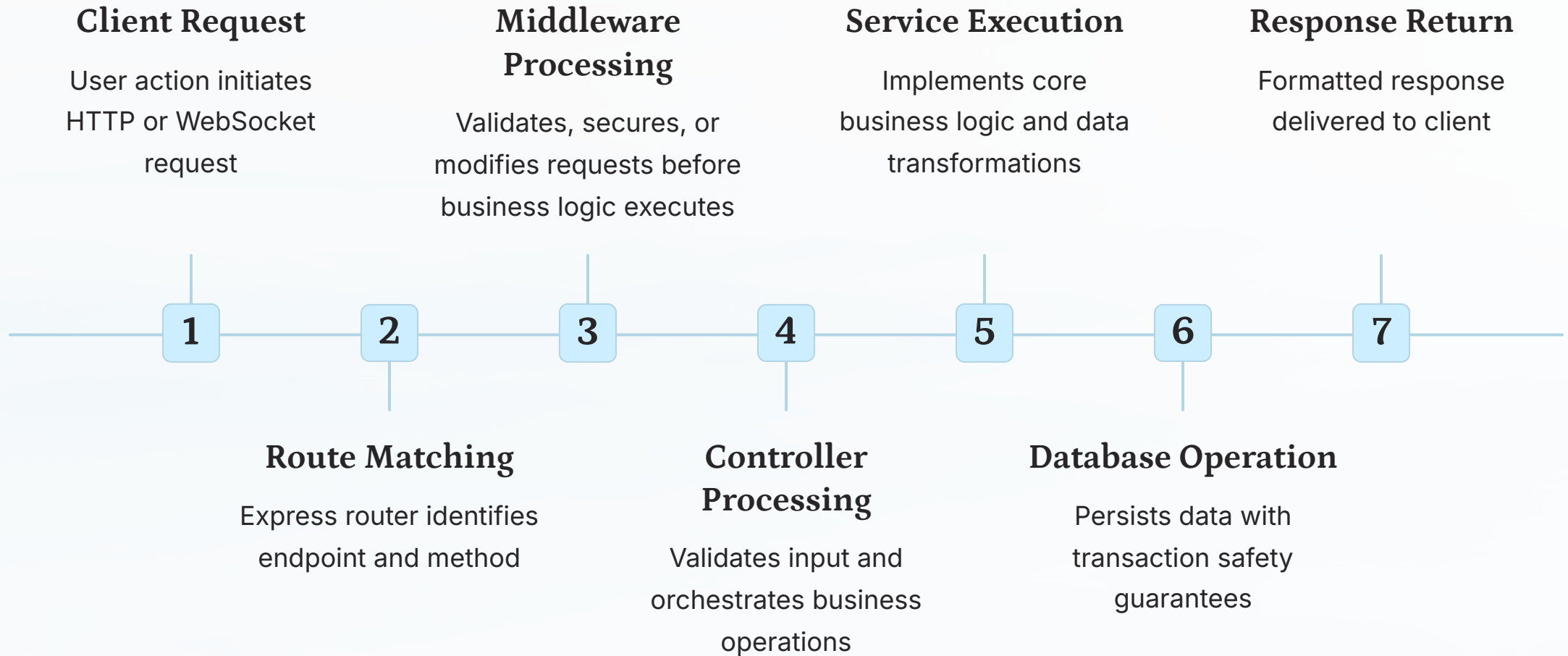
Utils

Contain reusable utility functions and helpers for common tasks across the application.

Config

Store environment-specific settings, constants, and external service configurations.

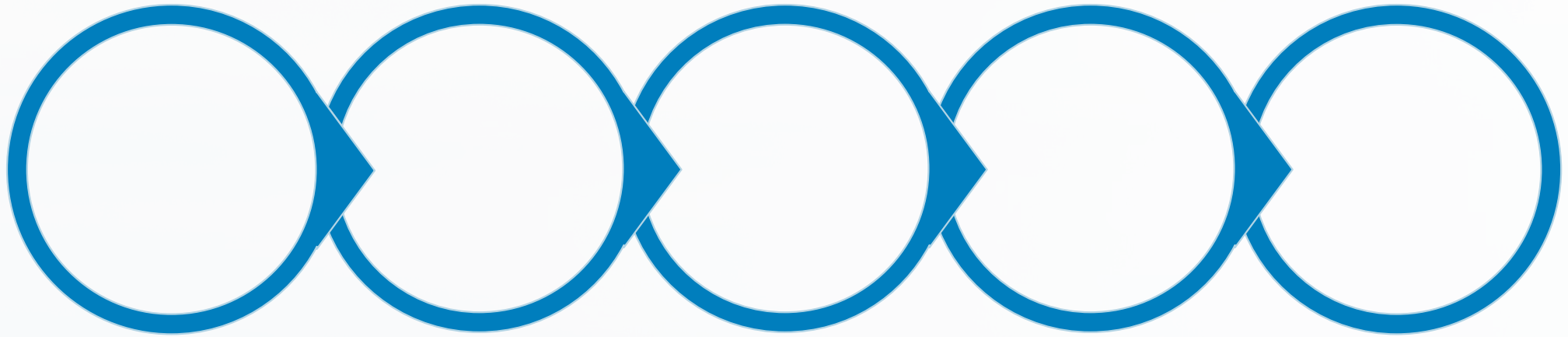
Request Flow



Request Flow

Component	Main Responsibility	When It Runs	Example Task	Focus
Route	Maps URL to handler	First	/api/chat/send → chatController	Navigation
Middleware	Validate / secure request	Before controller	Check JWT	Security / preprocessing
Controller	Handle request & response	After middleware	Read request body	Coordination
Service	Business logic	Inside controller	Send message logic	Core logic
Database	Store / retrieve data	Inside service	Insert message	Persistence
Response	Send result to client	Last	JSON success message	Output

Authentication System



The system uses both JWT and session-based authentication to support REST APIs and WebSocket connections.

Key Components

- JWT tokens
- Refresh tokens
- Sessions
- OTP verification

Token Explanation

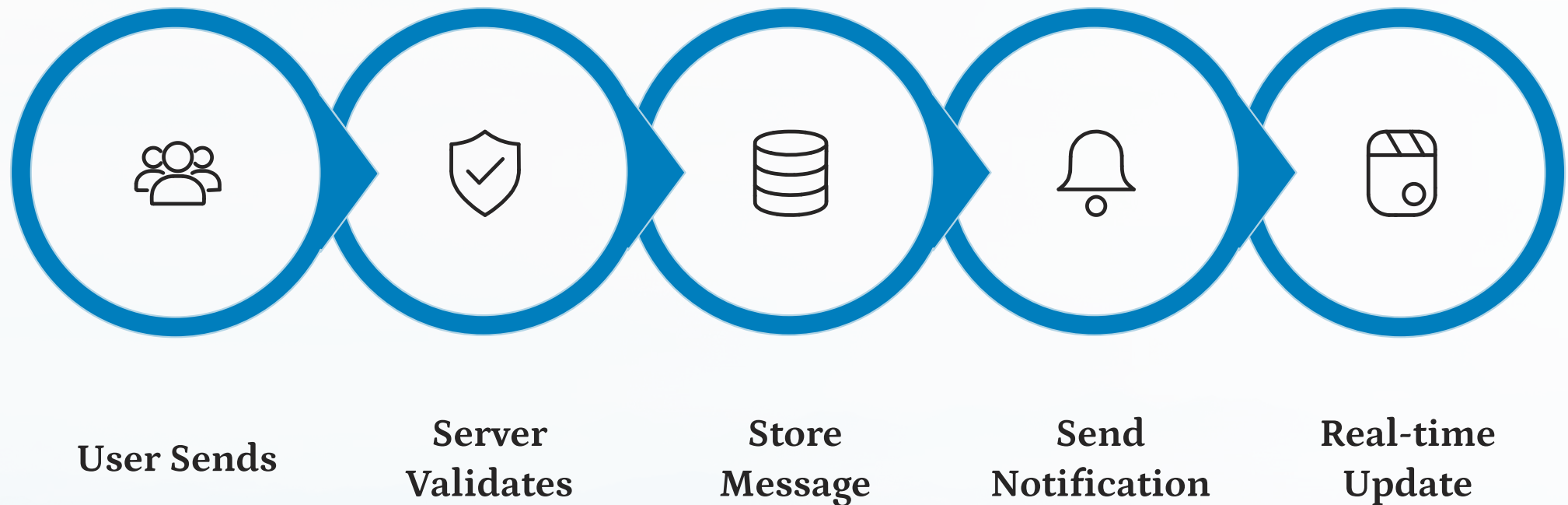
Access Token: Short-term authentication (expires in 15 minutes)

Refresh Token: Long-term authentication (expires in 7 days)

Session: Maintains login state in Redis

☐ The system uses both JWT and session-based authentication to support REST APIs and WebSocket connections.

Chat System Architecture



The message journey moves from the user to server validation, then database storage, notification, and finally real-time delivery to connected clients.

Key Features

- Message encryption
- Deduplication
- Transaction safety
- Real-time delivery

How It Works

Messages are inserted inside a database transaction, then real-time notifications are triggered after the transaction commits. This ensures data consistency and reliable delivery.

Socket.IO : Real Time Connections

Unlike traditional HTTP polling that repeatedly queries the server, WebSocket maintains persistent connections for instant bidirectional communication.

- **Instant Messaging**

Messages delivered in milliseconds

- **Online Status**

Live user presence tracking

- **Typing Indicators**

Real-time presence awareness

- **System Notifications**

Event-driven alerts and updates

Security, Scalability, Performance



Redis Caching

Session storage, message caches, presence data, and rate limiting counters reduce database load



Security Stack

Argon2 password hashing, JWT token rotation, audit logging, and comprehensive rate limiting



Connection Pooling

PostgreSQL connection pooling optimizes resource utilization under heavy load



User Protection

Blocking, reporting, message encryption, and content moderation features

Future Works (If pursued)

Microservices Architecture

Transition to a distributed system for enhanced resilience and independent scaling of services.

Kubernetes Deployment

Automated deployment, scaling, and management of containerized applications for operational efficiency.

Push Notifications

Implement real-time alerts to keep users engaged and informed of new messages or events.

AI Moderation

Leverage artificial intelligence for automated content filtering and enhanced user safety.

Message Search Optimization

Implement advanced indexing and search algorithms for lightning-fast message retrieval.

Modular for Scalability

Refactor codebase into smaller, more independent modules to support future growth and development.

